

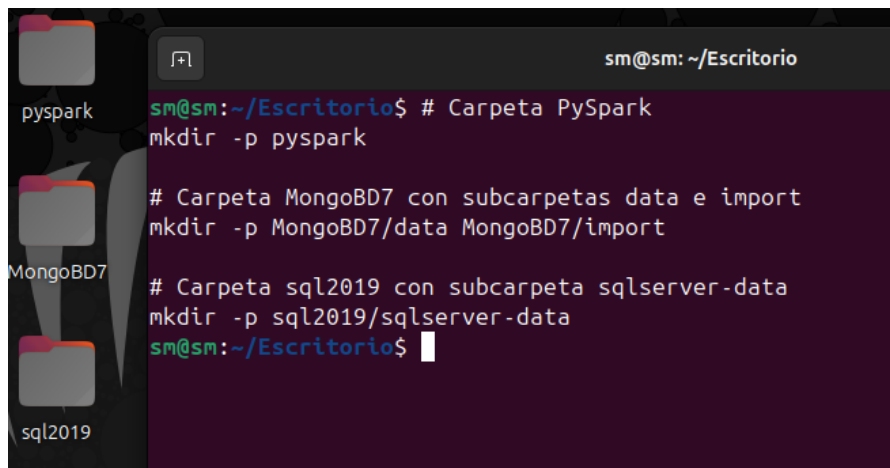
Lab 05 dirigido

Configuración Inicial

El presente ejercicio tiene lugar dentro de la maquina virtual basada en ubuntu

Crearemos las carpetas necesarias para simular 3 aspectos de negocio;

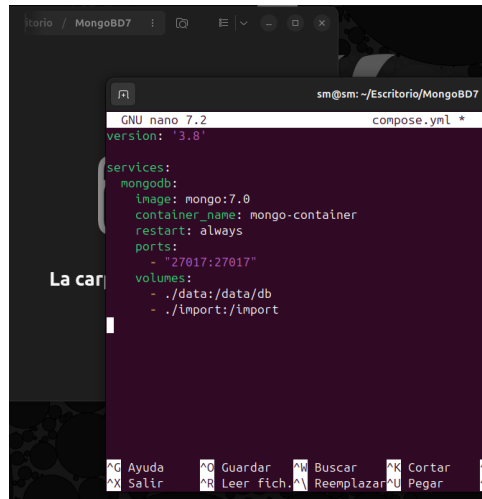
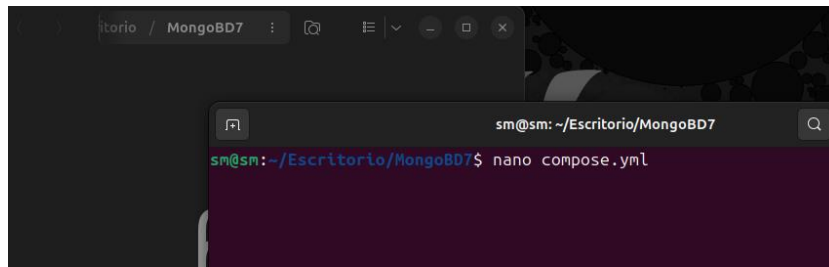
- **pyspark**, entorno de bigdata el cual se encargada del procesamiento masivo de datos dando como resultado los ficheros .parquet que luego formaran parte de una logica.
- **MongoBD**, simula la base de datos no estructurada la cual alberga la logica de municipalidades y sus temporadas.
- **SQL**, simula la logica correspondiente a la temporalidad; nota importante, si tiene instalado Managment Studio en su máquina principal (Windows) no es necesario instalar algún software adicional dado que la conexión se realizará por IP.

A screenshot of a terminal window with a dark background. The window title is 'sm@sm: ~/Escritorio'. The terminal shows three commands being executed to create directories. The first command creates 'pyspark'. The second command creates 'MongoBD7' and its subdirectories 'data' and 'import'. The third command creates 'sql2019' and its subdirectory 'sqlserver-data'. On the left side of the terminal, there are three folder icons labeled 'pyspark', 'MongoBD7', and 'sql2019'.

Carpeta pyspark mkdir -p pyspark
Carpeta MongoDB7 con subcarpetas data e import mkdir -p MongoDB7/data MongoDB7/import
Carpeta sql2019 con subcarpeta sqlserver-data mkdir -p sql2019/sqlserver-data

MongoDB

- En la carpeta de Mongo crearemos el archivo .yaml correspondiente



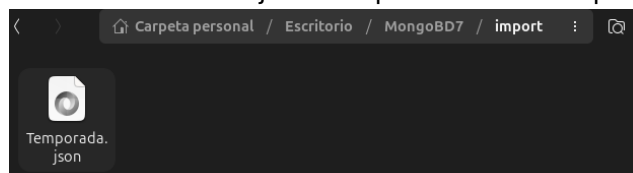
```
version: '3.8'

services:
  mongodb:
    image: mongo:7.0
    container_name: mongo-container
    restart: always
    ports:
      - "27017:27017"
    volumes:
      - ./data:/data/db
      - ./import:/import
```

Al iniciar el contenedor se vinculará con las siguientes carpetas:

```
sudo docker compose up -d
```

- Persistencia en ./data (en tu máquina).
- Carpeta ./import para colocar los ficheros y luego poder cargarlos.
- Colocar los ficheros con extensión .json compartidos en la carpeta import



- Cargar fichero json a MongoDB

```
sudo docker exec -it mongo-container mongoimport --db Municipalidades --collection Temporada --file /import/Temporada.json
```

- usa --jsonArray si tu JSON es un array []

```
sudo docker exec -it mongo-container mongoimport --db Municipalidades --collection Temporada --file /import/Temporada.json --jsonArray
```

****No aplica a este escenario**

```
Municipalidades 48.00 KiB
admin            40.00 KiB
config          108.00 KiB
local           72.00 KiB
test>

sm@sm:~/Escritorio/Mongo807/import$ sudo docker exec -it mongo-container mongoimport --db Municipalidades --collection Temporada --file /import/Temporada.json
2025-09-15T01:27:34.297+0000    connected to: mongod://localhost/
2025-09-15T01:27:34.345+0000    9 document(s) imported successfully. 0 document(s) failed to import.
```

- Puede validar los datos y/o esquemas cargados accediendo al mongosh

```
sudo docker exec -it mongo-container mongosh
```

```
mongosh mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.8

sm@sm:~/Escritorio$ sudo docker exec -it mongo-container mongosh
[sudo] contraseña para sm:
Current Mongosh Log ID: 68c76c08b9950a3653ce5f46
Connecting to: mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.8
Using MongoDB: 7.0.24
Using Mongosh: 2.5.8

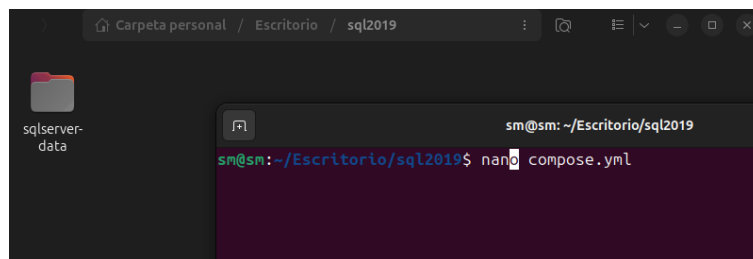
For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2025-09-15T00:23:02.245+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/production-file-system
2025-09-15T00:23:03.142+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2025-09-15T00:23:03.142+00:00: vm.max_map_count is too low
-----

test> show dbs
Municipalidades 136.00 KiB
admin            40.00 KiB
config          108.00 KiB
local           72.00 KiB
test> use Municipalidades
switched to db Municipalidades
Municipalidades> db.getCollectionNames()
[ 'Temporada', 'CategoriasMunicipalidades' ]
Municipalidades>
```

SQL Server 2019

- En la carpeta de SQL crearemos el archivo .yaml correspondiente



The screenshot shows a terminal window with the following content:

```
sm@sm:~/Escritorio/sql2019$ nano compose.yaml
```

The terminal window title is "sm@sm: ~/Escritorio/sql2019". The file explorer in the background shows the directory structure: "Carpeta personal / Escritorio / sql2019".

- Pegamos el código compartido y guardamos

```
sm@sm: ~/Escritorio/sql2019
GNU nano 7.2 compose.yml *
version: '3.8'

services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2019-latest
    container_name: sqlserver2019
    ports:
      - "14035:1433"
    environment:
      - ACCEPT_EULA=Y
      - MSSQL_SA_PASSWORD=Analitica
    volumes:
      - ./sqlserver-data:/var/opt/mssql
```

```
version: '3.8'

services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2019-latest
    container_name: sqlserver2019
    ports:
      - "14035:1433"
    environment:
      - ACCEPT_EULA=Y
      - MSSQL_SA_PASSWORD=Analitica
    volumes:
      - ./sqlserver-data:/var/opt/mssql
```

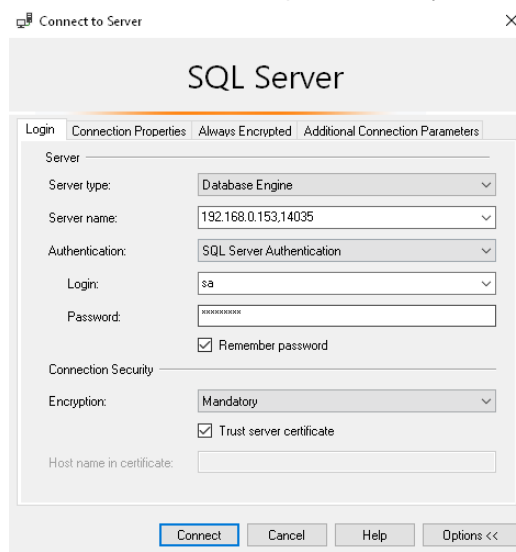
- Antes de ejecutar levantar el contenedor, ejecutar el siguiente comando para ajustar los permisos en la capeta:

```
sudo chown -R 10001:0 ./sqlserver-data
```

- Ahora ya puede inicializar el contenedor.

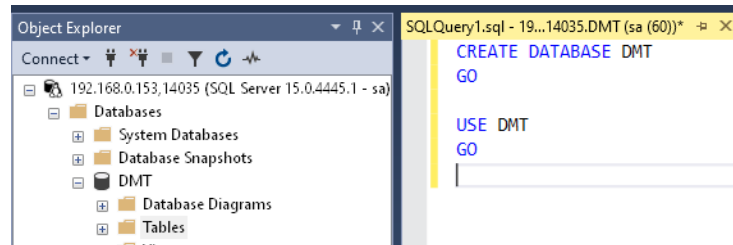
```
sudo docker compose up -d
```

- Una vez inicializado el contenedor, procedemos a acceder desde el Management Studio (local en Windows, no desde la maquina virtual).



** User: SA
 ** Pass: Anal1t1ca
 ** la IP varia dependiendo de su red
 ** puerto: 14035

- Procedemos a crear una base de datos denominada DMT



- Query representativo de la tabla:

```
CREATE TABLE dbo.DIM_FECHA (
  FechaKey      INT          NOT NULL PRIMARY KEY,
  Fecha         DATE         NOT NULL,
  Dia_Semana    TINYINT      NOT NULL,
  Nombre_Dia_Semana NVARCHAR(20) NOT NULL,
  Dia_Mes       TINYINT      NOT NULL,
  Dia_Anio      SMALLINT     NOT NULL,
  Nombre_Mes    NVARCHAR(20) NOT NULL,
  Mes_Anio      TINYINT      NOT NULL,
  Semana_Anio   TINYINT      NOT NULL,
  Anio          SMALLINT     NOT NULL,
  Bimestre      TINYINT      NOT NULL,
  Trimestre     TINYINT      NOT NULL,
  Cuatrimestre  TINYINT      NOT NULL,
  Semestre      TINYINT      NOT NULL,
  Fin_Semana    BIT          NOT NULL
);
```

**

- se adjunta un .sql con los datos respectivos para su población.

Transformación Archivos .Parquet

- En esta carpeta copiaremos los archivos (o toda la carpeta) contenidos en Origen el cual pertenece al ejercicio de Pyspark.
- También reutilizaremos la estructura del .yaml

```
services:
  transformers-networks:
    image: jupyter/pyspark-notebook
    ports:
      - 8000:8888
      - 4040:4040
    environment:
      - JUPYTER_TOKEN=desarrollo
    volumes:
      - ./:/home/jovyan
```

```
services:
  transformers-networks:
    image: jupyter/pyspark-notebook
    ports:
      - 8000:8000
      - 4040:4040
    environment:
      - JUPYTER_TOKEN=desarrollo
    volumes:
      - ./:/home/jovyan

sm@sm:~/Escritorio/pyspark$ sudo docker compose up -d
[sudo] contraseña para sm:
[+] Running 2/2
 ✓ Network pyspark_default          Created           0.1s
 ✓ Container pyspark-transformers-networks-1 Started        0.8s
sm@sm:~/Escritorio/pyspark$
```

- Luego procedemos a simular el flujo de negocio, el cual consta de la transformación en bloques de tipo parquet.

```
[1]: from pyspark.sql import SparkSession

# Crear sesión de Spark
spark = SparkSession.builder \
    .appName("DataLakeIngestion") \
    .getOrCreate()

[2]: # Configuración recomendada: tamaño de bloque/fichero ~128 MB
# (esto afecta a cómo Spark divide los datos al escribir)
spark.conf.set("spark.sql.files.maxPartitionBytes", 134217728) # 128 MB
spark.conf.set("spark.sql.parquet.compression.codec", "snappy") # típico en Data Lake

[3]: # Ruta base de tu Data Lake
base_path = "./datalake"

# Ruta base del Origen
base_origen = "./Origen"

Lectura de los archivos

[4]: # Leer CSVs
df_articles = spark.read.option("header", True).csv(f"{base_origen}/articles.csv")
df_customers = spark.read.option("header", True).csv(f"{base_origen}/customers.csv")

# Leer Parquet
df_transactions = spark.read.parquet(f"{base_origen}/transactions.parquet")

Escritura en formato Data Lake

[5]: # Guardar cada dataset en su carpeta
df_articles.write.mode("overwrite").parquet(f"{base_path}/articles")
df_customers.write.mode("overwrite").parquet(f"{base_path}/customers")
df_transactions.write.mode("overwrite").parquet(f"{base_path}/transactions")
```

**** Tenga en consideración que un flujo basado en actualizaciones requiere del uso de ejemplo: Airflow**

```
from pyspark.sql import SparkSession

# Crear sesión de Spark
spark = SparkSession.builder \
    .appName("DataLakeIngestion") \
    .getOrCreate()

# Configuración recomendada: tamaño de bloque/fichero ~128 MB
# (esto afecta a cómo Spark divide los datos al escribir)
spark.conf.set("spark.sql.files.maxPartitionBytes", 134217728) # 128 MB
spark.conf.set("spark.sql.parquet.compression.codec", "snappy") # típico en Data Lake
# Configuración recomendada: tamaño de bloque/fichero ~128 MB
# (esto afecta a cómo Spark divide los datos al escribir)
spark.conf.set("spark.sql.files.maxPartitionBytes", 134217728) # 128 MB
spark.conf.set("spark.sql.parquet.compression.codec", "snappy") # típico en Data Lake

# Ruta base de tu Data Lake
base_path = "./datalake"

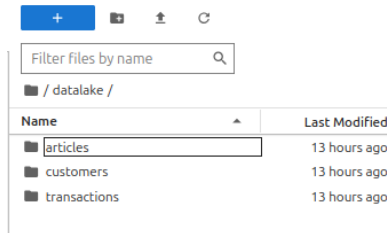
# Ruta base del Origen
base_origen = "./Origen"
```

```
# Leer CSVs
df_articles = spark.read.option("header", True).csv(f"{base_origen}/articles.csv")
df_customers = spark.read.option("header", True).csv(f"{base_origen}/customers.csv")

# Leer Parquet
df_transactions = spark.read.parquet(f"{base_origen}/transactions.parquet")

# Guardar cada dataset en su carpeta
df_articles.write.mode("overwrite").parquet(f"{base_path}/articles")
df_customers.write.mode("overwrite").parquet(f"{base_path}/customers")
df_transactions.write.mode("overwrite").parquet(f"{base_path}/transactions")
```

- Ejecutar este script dará como resultado la siguiente salida



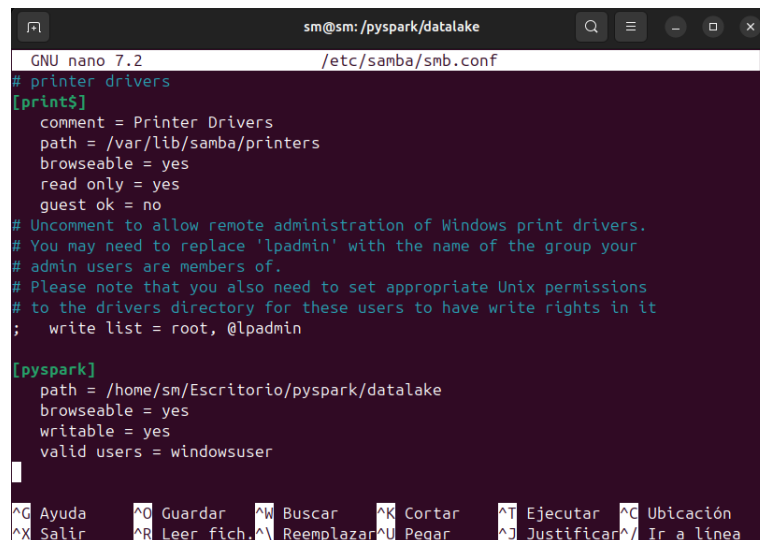
- Compartir vía Samba (SMB), hace que la carpeta aparezca en el Explorador de Archivos de Windows como si fuera una carpeta de red.

```
sudo apt update
sudo apt install samba -y

# Editar configuración de Samba
sudo nano /etc/samba/smb.conf
```

```
Created symlink /etc/systemd/system/nmb.service → /usr/lib/systemd/system/nmbd.service.
Created symlink /etc/systemd/system/multi-user.target.wants/nmbd.service → /usr/lib/systemd/system/nmbd.service.
Created symlink /etc/systemd/system/samba.service → /usr/lib/systemd/system/samba-ad-dc.service.
Created symlink /etc/systemd/system/multi-user.target.wants/samba-ad-dc.service → /usr/lib/systemd/system/samba-ad-dc.service.
Procesando disparadores para ufw (0.36.2-6) ...
Procesando disparadores para man-db (2.12.0-4build2) ...
Procesando disparadores para libc-bin (2.39-0ubuntu8.5) ...
sm@sn:~/Escritorio/pyspark$ sudo nano /etc/samba/smb.conf
```

Luego accedemos al documento .conf y usando el scroll del mouse procedemos a ir hasta el final del documento.



```
GNU nano 7.2 /etc/samba/smb.conf
# printer drivers
[print$]
    comment = Printer Drivers
    path = /var/lib/samba/printers
    browseable = yes
    read only = yes
    guest ok = no
# Uncomment to allow remote administration of Windows print drivers.
# You may need to replace 'lpadmin' with the name of the group your
# admin users are members of.
# Please note that you also need to set appropriate Unix permissions
# to the drivers directory for these users to have write rights in it
; write list = root, @lpadmin

[pyspark]
    path = /home/sm/Escritorio/pyspark/datalake
    browseable = yes
    writable = yes
    valid users = windowsuser
```

Al final del documento se agregarán las siguientes líneas:

```
[pyspark]
    path = /home/sm/Escritorio/pyspark/datalake
    browseable = yes
    writable = yes
    valid users = windowsuser
```

****Ctrl + o para guardar, enter para aceptar el nombre y Ctrl + x para salir**

****Tenga en consideración que la parte de la ruta “sm” corresponde al nombre de su maquina**

- Agregamos el usuario y el código nos solicitará la contraseña la cual será **datalake**

```
sudo adduser windowsuser
sudo smbpasswd -a windowsuser
sudo smbpasswd -e windowsuser
```

- Aseguramos los permisos de acceso a la ruta completa

```
sudo chmod 755 /home/sm
sudo chmod 755 /home/sm/Escritorio
sudo chmod -R 775 /home/sm/Escritorio/pyspark/datalake
```

****Tenga en consideración que la parte de la ruta “sm” corresponde al nombre de su maquina**

Realizar esta configuración puede resultar en un acierto en primera instancia, dado que ya tendrá las rutas compartidas; sin embargo, esto trae un problema del lado del contenedor el cual perdería el privilegio de escritura, para ello podemos dar permisos de escritura a todos, aunque resulta menos seguro.

```
sudo chmod -R 777 /home/sm/Escritorio/pyspark/datalake
```

Lo ideal sería emplear grupos de directorios para tener un mayor control sobre estos tipos de escenarios.

- Reiniciamos Samba

```
sudo systemctl restart smbd
```


- Desde Windows prueba con usuario Samba

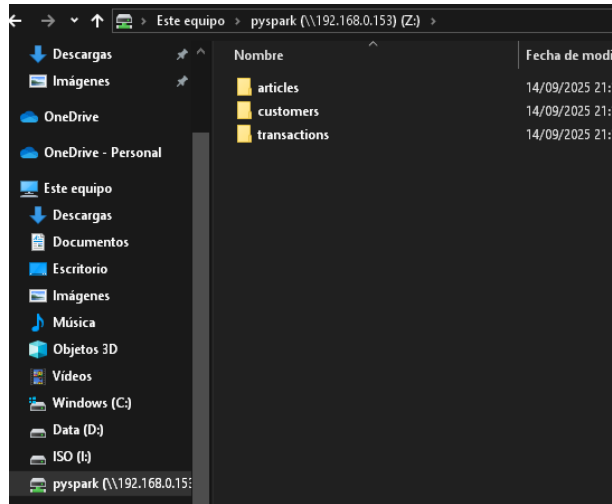
```
net use Z: \\192.168.0.153\pyspark /user:windowsuser datalake
```

```
C:\Users\Lenovo>net use Z: \\192.168.0.153\pyspark /user:windowsuser datalake
Se ha completado el comando correctamente.
```

```
C:\Users\Lenovo>
```

**** Tenga en consideración cambiar a la IP de su máquina virtual**

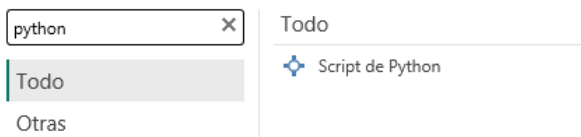
- Finalmente validamos el acceso a los datos



Power BI

- Conexión por script Python

Obtener datos



```
import pandas as pd
from pymongo import MongoClient

client = MongoClient("mongodb://192.168.0.153:27017")
db = client["Municipalidades"]
coll = db["Temporada"]

cursor = coll.find(
    {},
    {"_id": 0, "TemporadaID": 1, "Temporada": 1, "FechaInicio": 1, "FechaFin": 1}
)

DIM_TEMPORADA = pd.DataFrame(list(cursor))
DIM_TEMPORADA
```

```
import pandas as pd
from pymongo import MongoClient

client = MongoClient("mongodb://192.168.0.153:27017")
db = client["Municipalidades"]
coll = db["CategoriasMunicipalidades"]

cursor = coll.find(
    {},
    {"_id": 0, "Municipalidad": 1, "Categoria": 1}
)

DIM_CategoriasMunicipalidades = pd.DataFrame(list(cursor))
DIM_CategoriasMunicipalidades
```

- Conexión SQL Server

Obtener datos

Todo

Base de datos

Todo

Base de datos SQL Server

Base de datos SQL Server Analysis Services

Base de datos SQL Server

Servidor ⓘ

Base de datos (opcional)

Modo Conectividad de datos ⓘ

☒ Importar
 ☐ DirectQuery

► Opciones avanzadas

Aceptar

Cancelar

- Conexión a ficheros .parquet

1. Seleccionamos la opción “carpeta”

Todo

Archivo

Base de datos

Microsoft Fabric

Power Platform

Azure

Todo

Libro de Excel

Texto o CSV

XML

JSON

Carpeta

PDF

Parquet

2. Colocamos la ruta mediante IP y seleccionamos la opción Transformar Datos

Carpeta

Ruta a la carpeta

\\192.168.0.153\pyspark

Examinar...

Aceptar

Cancelar

\\192.168.0.153\pyspark

Content	Name	Extension	Date accessed	Date modified	Date created	Attributes	Folder Path
Binary	part-00000-79ec5744-62ec-4ff2-abf9-be3ca86ac099-c...	.crc	14/09/2025 21:11:48	14/09/2025 21:11:53	14/09/2025 21:11:48	Record	\\192.168.0.153\pyspark\articles\
Binary	part-00001-79ec5744-62ec-4ff2-abf9-be3ca86ac099-c...	.crc	14/09/2025 21:11:48	14/09/2025 21:11:52	14/09/2025 21:11:48	Record	\\192.168.0.153\pyspark\articles\
Binary	part-00002-79ec5744-62ec-4ff2-abf9-be3ca86ac099-c...	.crc	14/09/2025 21:11:48	14/09/2025 21:11:53	14/09/2025 21:11:48	Record	\\192.168.0.153\pyspark\articles\
Binary	part-00003-79ec5744-62ec-4ff2-abf9-be3ca86ac099-c...	.crc	14/09/2025 21:11:48	14/09/2025 21:11:51	14/09/2025 21:11:48	Record	\\192.168.0.153\pyspark\articles\
Binary	_SUCCESS.crc	.crc	14/09/2025 21:11:53	14/09/2025 21:11:53	14/09/2025 21:11:53	Record	\\192.168.0.153\pyspark\articles\
Binary	part-00000-79ec5744-62ec-4ff2-abf9-be3ca86ac099-c0...	.parquet	15/09/2025 11:20:30	14/09/2025 21:11:53	14/09/2025 21:11:53	Record	\\192.168.0.153\pyspark\articles\
Binary	part-00001-79ec5744-62ec-4ff2-abf9-be3ca86ac099-c0...	.parquet	15/09/2025 11:20:30	14/09/2025 21:11:52	14/09/2025 21:11:52	Record	\\192.168.0.153\pyspark\articles\
Binary	part-00002-79ec5744-62ec-4ff2-abf9-be3ca86ac099-c0...	.parquet	15/09/2025 11:20:30	14/09/2025 21:11:53	14/09/2025 21:11:53	Record	\\192.168.0.153\pyspark\articles\
Binary	part-00003-79ec5744-62ec-4ff2-abf9-be3ca86ac099-c0...	.parquet	15/09/2025 11:20:30	14/09/2025 21:11:51	14/09/2025 21:11:51	Record	\\192.168.0.153\pyspark\articles\
Binary	_SUCCESS		14/09/2025 21:11:53	14/09/2025 21:11:53	14/09/2025 21:11:53	Record	\\192.168.0.153\pyspark\articles\
Binary	part-00000-131630a0-818d-49ac-ad46-c41da73006a3-...	.crc	14/09/2025 21:11:53	14/09/2025 21:11:58	14/09/2025 21:11:53	Record	\\192.168.0.153\pyspark\customers\
Binary	part-00001-131630a0-818d-49ac-ad46-c41da73006a3-...	.crc	14/09/2025 21:11:53	14/09/2025 21:11:58	14/09/2025 21:11:53	Record	\\192.168.0.153\pyspark\customers\
Binary	part-00002-131630a0-818d-49ac-ad46-c41da73006a3-...	.crc	14/09/2025 21:11:53	14/09/2025 21:11:58	14/09/2025 21:11:53	Record	\\192.168.0.153\pyspark\customers\
Binary	part-00003-131630a0-818d-49ac-ad46-c41da73006a3-...	.crc	14/09/2025 21:11:53	14/09/2025 21:11:58	14/09/2025 21:11:53	Record	\\192.168.0.153\pyspark\customers\
Binary	_SUCCESS.crc	.crc	14/09/2025 21:11:58	14/09/2025 21:11:58	14/09/2025 21:11:58	Record	\\192.168.0.153\pyspark\customers\
Binary	part-00000-131630a0-818d-49ac-ad46-c41da73006a3-...	.parquet	15/09/2025 11:20:30	14/09/2025 21:11:58	14/09/2025 21:11:58	Record	\\192.168.0.153\pyspark\customers\
Binary	part-00001-131630a0-818d-49ac-ad46-c41da73006a3-...	.parquet	15/09/2025 11:20:30	14/09/2025 21:11:58	14/09/2025 21:11:58	Record	\\192.168.0.153\pyspark\customers\
Binary	part-00002-131630a0-818d-49ac-ad46-c41da73006a3-...	.parquet	15/09/2025 11:20:30	14/09/2025 21:11:58	14/09/2025 21:11:58	Record	\\192.168.0.153\pyspark\customers\
Binary	part-00003-131630a0-818d-49ac-ad46-c41da73006a3-...	.parquet	15/09/2025 11:20:30	14/09/2025 21:11:58	14/09/2025 21:11:58	Record	\\192.168.0.153\pyspark\customers\
Binary	_SUCCESS		14/09/2025 21:13:05	14/09/2025 21:11:58	14/09/2025 21:11:58	Record	\\192.168.0.153\pyspark\customers\

Los datos de la vista previa se han truncado debido a límites de tamaño.

Continuar

Cargar

Transformar datos

Cancelar

- Filtramos los datos preservando únicamente los ficheros de tipo parquet, y basado en la columna folder path seleccionamos 1 carpeta

```
let
    Origen = Folder.Files("\\192.168.0.153\pyspark"),
    #"Filas filtradas" = Table.SelectRows(Origen, each ([Extension] = ".parquet")),
    #"Columnas quitadas" = Table.RemoveColumns(#"Filas filtradas",{"Name", "Extension", "Date accessed", "Date modified", "Date created", "Attributes"}),
    #"Filas filtradas1" = Table.SelectRows(#"Columnas quitadas", each ([Folder Path] = "\\192.168.0.153\pyspark\articles\"))
in
    #"Filas filtradas1"
```

Consultas [6]

DIM_TEMPORADA

DIM_FECHA

DIM_CategoriasMunicipalidades

pyspark

fx

= Table.SelectRows(#"Columnas quitadas", each ([Folder Path] = "\\192.168.0.153\pyspark\articles\"))

	Content	Folder Path
1	Binary	\\192.168.0.153\pyspark\articles\
2	Binary	\\192.168.0.153\pyspark\articles\
3	Binary	\\192.168.0.153\pyspark\articles\
4	Binary	\\192.168.0.153\pyspark\articles\

- Este paso lo replicamos para cada carpeta y renombramos el nombre de estas:

Consultas [6]

DIM_TEMPORADA

DIM_FECHA

DIM_CategoriasMunicipalidades

DIM_ARTICLES

DIM_CUSTOMERS

DIM_TRANSACTIONS

fx

= Table.SelectRows(#"Columnas quitadas", each ([Folder Path] = "\\192.168.0.153\pyspark\articles\"))

	Content	Folder Path
1	Binary	\\192.168.0.153\pyspark\articles\
2	Binary	\\192.168.0.153\pyspark\articles\
3	Binary	\\192.168.0.153\pyspark\articles\
4	Binary	\\192.168.0.153\pyspark\articles\

- Borramos la columna folder path y expandimos el content el cual nos dira que son archivos parquet

Consultas [6]	Table.RemoveColumns(#"Filas filtradas1",{"Folder Path"})
- DIM_TEMPORADA - DIM_FECHA - DIM_CategoríasMunicipalidades - DIM_ARTICLES - DIM_CUSTOMERS - DIM_TRANSACTIONS	Content - Binary - Binary - Binary - Binary

article_id	product_code	prod_name	product_type_no	product_type_name	product_group_name	graphical_appearance_no
0108775015	0108775	Strap top	253	Vest top	Garment Upper body	1010016
0108775044	0108775	Strap top	253	Vest top	Garment Upper body	1010016
0108775051	0108775	Strap top (1)	253	Vest top	Garment Upper body	1010017
0110065001	0110065	OP T-shirt (Idro)	306	Bra	Underwear	1010016
0110065002	0110065	OP T-shirt (Idro)	306	Bra	Underwear	1010016
0110065011	0110065	OP T-shirt (Idro)	306	Bra	Underwear	1010016
0111565001	0111565	20 den 1p Stockings	304	Underwear Tights	Socks & Tights	1010016
0111565003	0111565	20 den 1p Stockings	302	Socks	Socks & Tights	1010016
0111586001	0111586	Shape Up 30 den 1p Tights	273	Leggings/Tights	Garment Lower body	1010016
0111593001	0111593	Support 40 den 1p Tights	304	Underwear Tights	Socks & Tights	1010016
0111609001	0111609	200 den 1p Tights	304	Underwear Tights	Socks & Tights	1010016

6. Finalmente tendremos todas las rutas basadas en ficheros parquet

Consultas [18]

Transformar archivo de DIM_ARTICLES [2]

Consultas auxiliares [3]

Transformar archivo de DIM_CUSTOMERS [2]

Consultas auxiliares [3]

Transformar archivo de DIM_TRANSACTIONS [2]

Otras consultas [6]

DIM_TEMPORADA

DIM_FECHA

DIM_CategoríasMunicipalidades

DIM_ARTICLES

DIM_CUSTOMERS

DIM_TRANSACTIONS

Table.TransformColumnTypes(#"Columna de tabla expendida",{"{t_dat, type date}", {"customer_id", type text}, {"article_id, Int64.Type}, {"price, type number}, {"p2 sales_channel_id, type text})

1

20/08/2018

00008a120b4b6e7c226688a19b18135c232f0a08f7b4e40...

687713003

0.03080508

2

2

20/08/2018

00008a120b4b6e7c226688a19b18135c232f0a08f7b4e40...

542518023

0.030492525

2

3

20/08/2018

00007c32b2e79b05483b024c612e66842531a6699338c357091...

505222004

0.03237288

2

4

20/08/2018

00007c32b2e79b05483b024c612e66842531a6699338c357091...

685687003

0.030892203

2

5

20/08/2018

00007c32b2e79b05483b024c612e66842531a6699338c357091...

685687004

0.030892203

2

6

20/08/2018

00007c32b2e79b05483b024c612e66842531a6699338c357091...

685687001

0.030892203

2

7

20/08/2018

00007c32b2e79b05483b024c612e66842531a6699338c357091...

505222002

0.030522004

2

8

20/08/2018

00008a041544b7b0b0d0c2905a4176a7f10075269b4732356c0b...

688873012

0.030492525

2

9

20/08/2018

00008a041544b7b0b0d0c2905a4176a7f10075269b4732356c0b...

505222001

0.032525424

2

10

20/08/2018

00008a041544b7b0b0d0c2905a4176a7f10075269b4732356c0b...

508859003

0.045745763

2

11

20/08/2018

00008a041544b7b0b0d0c2905a4176a7f10075269b4732356c0b...

688873001

0.030492525

2

12

20/08/2018

00008a041544b7b0b0d0c2905a4176a7f10075269b4732356c0b...

688873012

0.030492525

2

13

20/08/2018

00008a041544b7b0b0d0c2905a4176a7f10075269b4732356c0b...

528842001

0.030522004

2

14

20/08/2018

00008a041544b7b0b0d0c2905a4176a7f10075269b4732356c0b...

508859003

0.045745763

2

15

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

508859003

0.030892203

2

16

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

508859003

0.030892203

2

17

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

18

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

19

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

20

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

21

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

22

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

23

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

24

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

25

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

26

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

27

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

28

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

29

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

30

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

31

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

32

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

33

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

34

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

35

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

36

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

37

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

38

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

39

20/08/2018

0000a7f0b0c6c07174389b76c132a6786055f9f70699a0c14425...

671050001

0.030892203

2

Configuración de la consulta

PROPIEDADES

Nombre

DIM_TRANSACTIONS

Todas las propiedades

PASOS APLICADOS

Origen

Filas filtradas

Columnas quitadas

Filas filtradas1

Columnas quitadas1

Archivos ocultos filtrados1

Invozar función personalizada1

Otras columnas quitadas1

Columna de tabla expendida1

Pro Tipo cambiado

7. Se muestran las relaciones empleadas en el archivo PowerBI.

Administrar relaciones

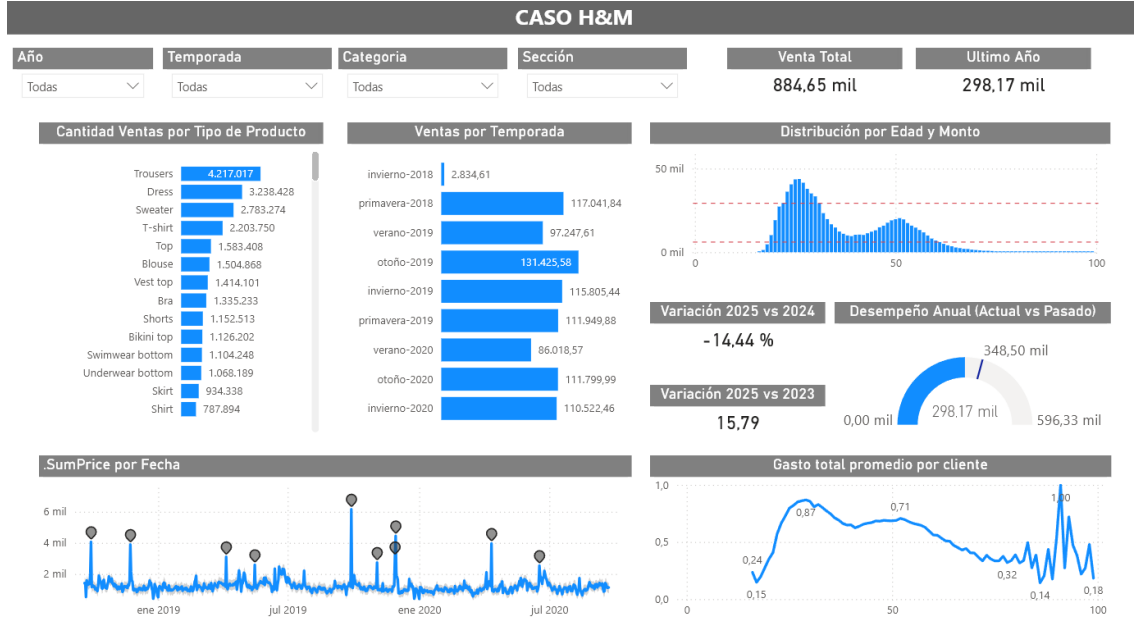
	Nueva relación		Detección automática		Editar		Eliminar		Filtro
☐	Desde: tabla (columna)	↑	Relación	A: tabla (columna)	Estado				
☐	DIM_TRANSACTIONS (article_id)			DIM_ARTICLES (article_id)	Activo	...			
☐	DIM_TRANSACTIONS (custom...			DIM_CUSTOMERS (customer_id)	Activo	...			
☐	DIM_TRANSACTIONS (t_dat)			DIM_FECHA (Fecha)	Activo	...			
☐	DIM_TRANSACTIONS (Tempor...			DIM_TEMPORADA (Temporad...	Activo	...			

La relación entre la DIM_TEMPORADA Y TRANSACTIONS depende de la creación de la columna calculada:

```
TemporadaID =
VAR fechaTrans = DIM_TRANSACTIONS[t_dat]
RETURN
MAXX(
    FILTER(
        'DIM_Temporada',
        fechaTrans >= 'DIM_Temporada'[FechaInicio]
        && fechaTrans <= 'DIM_Temporada'[FechaFin]
    ),
```

```
'DIM_Temporada' [TemporadaID]
)
```

- A continuación, se muestra un ejemplo de los graficos en Power BI



Mettricas	Origen
.SumPrice = SUM(DIM_TRANSACTIONS[price])	DIM_TRANSACTIONS
.CountCustomers = COUNTA(DIM_TRANSACTIONS[customer_id])	DIM_TRANSACTIONS
.ANIO_Actual = TOTALYTD([.SumPrice], DIM_FECHA[Fecha])	DIM_TRANSACTIONS
.ANIO_Pasado = TOTALYTD([.SumPrice], DATEADD (DIM_FECHA[FECHA], -1, YEAR))	DIM_TRANSACTIONS
.ANIO_Pasado -2 = TOTALYTD([.SumPrice], DATEADD (DIM_FECHA[FECHA], -2, YEAR))	DIM_TRANSACTIONS
.%VarPrice = DIVIDE([.ANIO_Actual] - [.ANIO_Pasado], [.ANIO_Pasado])	DIM_TRANSACTIONS
.%VarPrice -2 = DIVIDE([.ANIO_Actual] - [.ANIO_Pasado -2], [.ANIO_Pasado -2])	DIM_TRANSACTIONS
.TituloVarPrice = "Variación " & YEAR(TODAY()) & " vs " & YEAR(TODAY())-1	DIM_TRANSACTIONS
.TituloVarPrice_2 =	DIM_TRANSACTIONS

"Variación " & YEAR(TODAY()) & " vs " & YEAR(TODAY())-2	
.DistinctCountCustomers = DISTINCTCOUNT(DIM_TRANSACTIONS[customer_id])	DIM_TRANSACTIONS
.Avg_spent_per_customer = DIVIDE([.SumPrice], [.DistinctCountCustomers])	DIM_TRANSACTIONS
TemporadaID = VAR fechaTrans = DIM_TRANSACTIONS[t_dat] RETURN MAXX(FILTER('DIM_Temporada', fechaTrans >= 'DIM_Temporada'[FechaInicio] && fechaTrans <= 'DIM_Temporada'[FechaFin]), 'DIM_Temporada'[TemporadaID])	